

Teknologi

Tråde – del 2

Dagsorden

- Synkroniseringsmekanismer
- Flere problemer med tråde
 - Deadlock
 - Livelock
 - Starvation

SYNKRONISERINGSMEKANISMER

Første problem med tråde!

- **Hvad kan gå galt?**
 - Race conditions
- **Hvorfor går det galt?**
 - Fordi trådene kan "pille" ved delte ressourcer
 - Fordi ikke alle operationerne ikke er atomare!
- **Hvordan forhindrer vi det går galt?**
 - Petersons algoritme men den havde begrænsninger
 - Særligt mht. skalerbarhed
 - En moderne og mere fleksibel tilgang er synkroniseringsmekanismer

Synkroniseringsprimitiver i .NET

- Gør brug af et "låse" begreb
 - Eksklusive låse
 - Garanterer gensidig udelukkelse
 - Dvs. kun en tråd ad gangen
 - Fx: Monitor, Mutex
 - Andre låse
 - Styring af mængden eller formålet af tråde i en kritiske region
 - Fx : Semaphore, ReaderWriterLock
 - <https://docs.microsoft.com/en-us/dotnet/standard/threading/overview-of-synchronization-primitives>
 - Bemærk : At der findes mange flere end dem vi kan nå at gennemgå i undervisningen!

Lock (1/2)

den låses når en tråd går lock(), og låses op igen, når tråden går ud igen

de andre tråde får en "time out" fra at udføre resten af koden mens den første tråd er inde i lock

- Lock er et **keyword** i C#
 - Det kan bruges til at skabe en "lås" på et stykke kode (en kritisk region)
 - Dette betyder at kun en enkelt tråd vil kunne befinde sig inde i den kritiske del af koden ad gangen
 - Den virker ikke på tværs af processer
- Lock bruger et "låse" objekt
 - Den tråd der har låsen har adgang til den kritiske region
 - Det er vigtigt at alle tråde bruger samme instans af låse objektet – derfor er disse ofte placeret i statiske variable
- Tråde der venter på at få "låsen" vil blive blokeret
 - Dvs. de sover og bruger derfor ikke CPU resurser
- Note: Lock bruger internt en anden synkroniseringsmekanisme : monitor

Lock (2/2)

```
//State er en delt resurse
static int state = 5;
static readonly object lockObject = new object(); //Objekt der bliver brugt til at låse på
static void Main(string[] args)
{
    //Opretter og starter 2 tråde
    for (int i = 0; i < 2; i++)
    {
        Thread t = new Thread(RunMe);

        t.Start();
    }
}

static void RunMe()
{
    int i = 0; //Anvendes til at finde ud af hvornår race condition opstår
    while (true)
    {
        lock(lockObject)
        {
            if (state == 5) // Kontrollere om state er 5
            {
                state++; //Tæller state 1 op
                Trace.Assert(state == 6, "Race condition in loop " + i); //Tjekker om state er 6, hvis ikke = fejl
            }
            state = 5; // Nulstiller state til 5
        }
        i++;
    }
}
```

Låseobjektet er her statisk således det er garanteret alle tråde bruger samme objekt instans samt readonly således den ikke kan ændres

Dette er den kritisk region, da det er her inden for der bliver læst og manipuleret med delt data.

Note: det er vigtigt ikke at låse mere end nødvendigt at det begrænser trådene mulig for at køre frit!

Øvelser

øvelse 04 a

- Ved brug af forrige slide, løs problemet fra i går
konklusion får ikke en racecondition
- Læs en forklaring af "lock" nøgleordet på nedenstående hjemmeside og forklar det for hinanden i grupper 2 og 2
 - <http://www.dotnetperls.com/lock>

Monitor (1/3)

- **Monitor - I C# er det en klasse**
 - “Provides a mechanism that synchronizes access to objects” står der i dokumentationen på <https://msdn.microsoft.com/>

Monitor (2/3)

- Man kan se den som en slags udvidelse af lock keywordet
 - Generelt anbefales det dog at benytte lock med mindre man har brug for større kontrol / styring med låsen
- I stedet for lock kalder man `Monitor.Enter()` og `Monitor.Exit()`

```
10 public class Program
11 {
12     static decimal balance = 50;
13     static Object thisLock = new Object(); // Object that is going to be locked
14     static void Main(string[] args)
15     {
16         for (int i = 1; i <= 5; i++) // Create and start 5 threads
17         {
18             Thread t = new Thread(new ParameterizedThreadStart(Withdraw));
19             t.Start(15);
20         }
21         Console.ReadKey();
22     }
23     static void Withdraw(object obj) // Passing data
24     {
25         int amount = (int)obj; // Casting the data to correct type
26         Monitor.Enter(thisLock); // Mutex on an object
27         try
28         {
29             if (amount > balance)
30             {
31                 Console.WriteLine("Insufficient funds");
32                 return;
33             }
34             else
35             {
36                 balance -= amount;
37                 Console.WriteLine(balance);
38             }
39         }
40         finally
41         {
42             Monitor.Exit(thisLock); // Releasing the mutex
43         }
44     }
45 }
```

```
35
20
5
Insufficient funds
Insufficient funds
```

Monitor (3/3)

- Man kan se den som en slags udvidelse af lock keywordet, idet alle lock(obj) er implementeret ved brug af Monitor.
- I stedet for lock kalder man Monitor.Enter() og Monitor.Exit()
- Man kan ikke skabe instanser af Monitor klassen (statisk)
- Alle lock statements kompiles til de statiske metoder Monitor.Enter() og Monitor.Exit() + try-finally

Det synkroniserende objekt

- Når lock eller monitor bruges skal den sættes på et objekt men hvilket(?)
- Det skal blot være et objekt som alle trådene kan se, og samtidigt skal det være en reference type. Typisk er dets visibility/access modifier privat
- Er den delte resurser i sig selv et objekt (variabel med en reference type) kan denne i sig selv bruges som delingsobjekt
 - Fx. Kan den delte resurse være en liste flere tråde tilgår
 - Men vær opmærksom på at alle tråde bruger samme objekt!
 - Problemer kan opstå hvis selve objektet overskrives



Ændre indholdet (Fx. `myList.Add(10)`) af en liste er ofte ok (I bør dog bruge et dedikeret låseobjekt)



Overskrive listen (Fx `myList = new List<int>()`) er ofte ikke ok, når trådene først er sat i gang
Her skal der i så fald bruges et ekstra objekt til synkroniseringen alene (fx `object syncMyList = new object()`)

Mutex (1/7)

det kunne fx være, at man ikke kan starte 2 af det samme spil på samme tid. Spil nr 2 vil blive lukket ned med det samme

- **Mutex**
 - Hvad er Mutex?
 - I C# er det en klasse
 - “A synchronization primitive that can also be used for interprocess synchronization” står der i dokumentationen på <https://msdn.microsoft.com/>

Mutex (2/7)

- **Mutex**

- Man kan se den som en lock/lås
- Den kan virke på tværs af processer i OS
 - Findes som lokal og named(globale for OS).
 - Named tager imod et navn og eksisterer på operativsystemniveau, så den kan bruges af flere processer ikke kun af det program vi skaber
 - Hint: Kan faktisk udnyttes på en smart måde til at identificere om et program allerede at startet
- Hvis en tråd har fået adgang, bliver den næste der spørger blokeret indtil resursen er fri igen
- **Mini øvelse: Google "C# thread states" og find og diskuter et billede med en medstuderende**

Mutex (3/7)

- **Mutex**

- En mutex kan kun frigives af den tråd der fik den
 - Man siger den "enforces thread identity"
- Hvis tråden der har mutex'en dræbes eller crasher inden mutex'en frigives, siger man den er "abandoned". I dette tilfælde er det den næste tråd, der beder om adgang som får den
- Bemærk :
 - At en abandoned mutex er en fejl situation og derfor kaster .NET en exception AbandonedMutexException i den nye tråd.
 - Dette kan dog så gribes og håndteres. Læs selv mere her : <https://docs.microsoft.com/en-us/dotnet/api/system.threading.abandonedmutexexception?view=net-6.0>
 - I de fleste programmeringssprog vil en "abandoned" lås betyde risiko for en deadlock situation (forklares senere)

Mutex (4/7)

- **Brugen af Mutex**

- Vi bruger metoden `WaitOne()` for at bede om låsen
- Vi bruger `ReleaseMutex()` til at give låsen fri
 - Når der står at den virker indenfor OS betyder det at man kan kalde den fra en anden proces.

Mutex (5/7)

```
//State er en delt resurse  
static int state = 5;  
static Mutex m = new Mutex(); //Mutex der bliver brugt til at låse på  
static void Main(string[] args)
```

Oprettelse af mutex

```
{  
  
    //Opretter og starter 2 tråde  
    for (int i = 0; i < 2; i++)  
    {  
        Thread t = new Thread(RunMe);  
  
        t.Start();  
    }  
}
```

```
static void RunMe()  
{
```

```
    int i = 0; //Anvendes til at finde ud af hvornår race condition opstår  
    while (true)
```

Indkapsling af den kritiske region

```
{  
    m.WaitOne();  
    if (state == 5) // Kontrollere om state er 5  
    {  
        state++; //Tæller state 1 op  
        Trace.Assert(state == 6, "Race condition in loop " + i); //Tjekker om state er 6, hvis ikke = fejl  
        Console.WriteLine("."); //Tilføjet så man kan se der sker noget :-)  
    }  
    state = 5; // Nulstiller state til 5  
    m.ReleaseMutex();  
    i++;  
}
```

Mutex (6/7)

- Særlige muligheder med Mutex

- Det er muligt at sige man kun venter et bestemt stykke tid på at få mutex'en

```
if (m.WaitOne(1))  
{  
    if (state == 5) // Kontrollere om state er 5  
    {  
        state++; //Tæller state 1 op  
        Trace.Assert(state == 6, "Race condition in loop " + i); //Tjekker om state er 6, hvis ikke = fejl  
        Console.WriteLine("."); //Tilføjet så man kan se der sker noget :-)  
    }  
    state = 5; // Nulstiller state til 5  
    Thread.Sleep(10);  
    m.ReleaseMutex();  
}  
else  
{  
    Console.WriteLine("Did not get mutex");  
}
```

Her ventes der 1 millisekund på at få låsen
Hvis ikke returnere den false og man kan
reagere tilsvarende

Hint : Det er dette princip, der i kombination med
en navngivet mutex, kan bruges til at identificere
om samme program allerede kører på en computer.
Og en eventuel ny instans af programmet kan så
lukkes ned i else blokken

Mutex (7/7)

- Særlige muligheder med Mutex

- En mutex kan oprettes til at være ejet på forhånd

- Dette kan bruges til at bestemme hvornår en tråd aktiveres:

Mutex er taget på forhånd

```
//State er en delt resurse
static int state = 5;
static Mutex m = new Mutex(true); //Mutex der bliver brugt til at låse på
static void Main(string[] args)
{
    //Opretter og starter 2 tråde
    for (int i = 0; i < 2; i++)
    {
        Thread t = new Thread(RunMe);

        t.Start();
    }

    Console.WriteLine("Threads are waiting - Press any key to start them");
    Console.ReadLine();
    m.ReleaseMutex();
}

static void RunMe()
{
    int i = 0; //Anvendes til at finde ud af hvornår race condition opstår
    while (true)
    {
        m.WaitOne();

        if (state == 5) // Kontrollere om state er 5
        {
            state++; //Tæller state 1 op
            Trace.Assert(state == 6, "Race condition in loop " + i); //Tjekker om state er 6, hvis ikke = fejl
            Console.Write("."); //Tilføjet så man kan se der sker noget :-
        }
        state = 5; // Nulstiller state til 5
        m.ReleaseMutex();

        i++;
    }
}
```

Her frigives mutex'en og
trådene slippes løs

Øvelser

øvelse 04 b

- **Løs raceconditionen fra øvelsen i går ved brug af mutex**
 - Prøv også at lave en løsning hvor der kun ventes et stykke tid på at få en lås
- **Prøv at lave en start linje løsning**
 - Dvs. brug en lås til at slippe en tråd løs

Semaphore (1/4)

- I C# er det en klasse som hedder Semaphore og som...
 - “Limits the number of threads that can access a resource or pool of resources concurrently”
- Styre altså mængden af tråde der kan være i en kritisk region

Semaphore (2/4)

- Semaphore

- Det virker lidt svært at forstå, så tænk på en semaphore som en natklub, med en bestemt kapacitet.
 - Når den er fyldt står gæsterne i kø udenfor
 - Først når en gæst forlader klubben kan en ny komme ind
- Note: En tråd kan gå ind i en semaphore flere gange (Hvis WaitOne kaldes flere gange af samme tråd)
- Konstruktøren kræver to argumenter: aka hvor mange ledige pladser der er i klubben og hvor stor maksimal kapaciteten er
den skal tænkes omvendt fx: int ledige pladser, int max kapaciteten
- En semaphore med plads til én svarer til mutex/lock, bortset fra at den ikke har en ejer, så enhver tråd kan kalde release på den dvs. ingen tvunget thread identity

Semaphore (3/4)

- Semaphore

```
10 class NightClub
11 {
12     (mySemaphore)
13     static Semaphore MySemaphore = new Semaphore(0,3); // Initial count 0, max capacity of 3
14     static void Main()
15     {
16         for (int i = 1; i <= 5; i++)
17         {
18             new Thread(Enter).Start(i);
19         }
20         Thread.Sleep(500); // Main thread sleeps to allow threads to start and block the semaphore
21         Console.WriteLine("Main thread calls Release(3).");
22         MySemaphore.Release(3); // Main thread releases the semaphore e.g. NightClub is opening
23         Console.ReadKey();
24     }
25     static void Enter(object id)
26     {
27         Console.WriteLine(id + " Starts and waits outside to enter");
28         MySemaphore.WaitOne(); // Only three threads in here!
29         Console.WriteLine(id + " Enters the Nightclub!");
30         Thread.Sleep(1000 * (int)id);
31         Console.WriteLine(id + " is leaving");
32         MySemaphore.Release();
33     }
34 }
```

```
4 Starts and waits outside to enter
1 Starts and waits outside to enter
3 Starts and waits outside to enter
5 Starts and waits outside to enter
2 Starts and waits outside to enter
Main thread calls Release(3).
1 Enters the Nightclub!
3 Enters the Nightclub!
4 Enters the Nightclub!
1 is leaving
5 Enters the Nightclub!
3 is leaving
2 Enters the Nightclub!
4 is leaving
2 is leaving
5 is leaving
```

Semaphore (4/4)

- Semaphore

- Vi bruger constructoren med to argumenter
- Vi bruger metoden `.WaitOne()`, til at spørge om adgang
- Vi bruger metoden `.Release()`, til at forlade
- Husk at count tælles ned hver gang en tråd får adgang. Så når den er 0, er der tre tråde inde og nye tråde må derfor vente. Når en tråd forlader tælles count op.
- `Release()` frigiver én plads, svarende til at en enkelt tråd forlader den kritiske region
- `Release(x)` frigiver det antal der skrives som argument
 - `Release(3)` kan derved sørge for at alle kommer ud, dvs. tømme natklubben.

FLERE PROBLEMER MED TRÅDE

Problemer med tråde

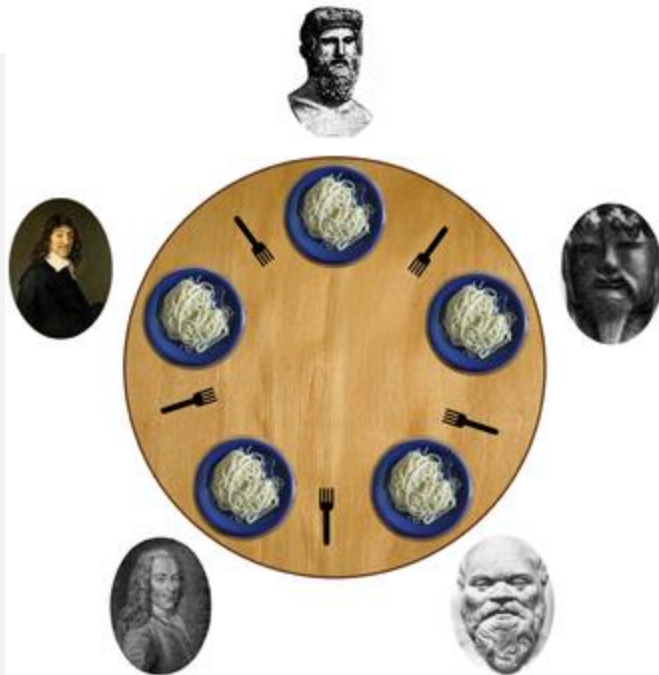
- **Hvad kan gå galt?**

- Vi undgår nu race conditions, fordi vi har en masse forskellige synkroniseringsmekanismer, til gengæld kan vi få
 - Dead locks
 - Live locks
 - Starvation

- **Hvorfor går det galt?**

- Fordi trådene vha. mekanismerne kan blokkere for hinanden!!!

Dining philosophers



Forudsætninger :

1. En gaffel per person
2. Det kræver 2 gafler at spise spaghetti
 - Gaflerne bliver altså en delt resurse
3. Hver filosof kan agere uafhængigt af hinanden
 - Altså er de "tråde"

Problemet :

Hvis alle filosofer reagere samtidig tager de alle én gaffel hver -> de ender op med at skulle vente til en anden filosof slipper deres gaffel

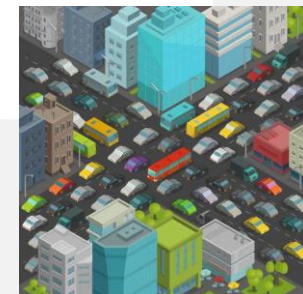
Men hvem vil det ?

— Er et klassisk eksempel på deadlocks

— Se også: https://en.wikipedia.org/wiki/Dining_philosophers_problem

Deadlocks

- Vi så "The Dining philosophers" eksemplet
- En deadlock er altså en tilstand hvor to eller flere tråde gensidigt venter på frigivelsen af ressourcer de andre tråde har
- Da de begge venter på hinandens ressourcer ender de med at stå og låse hinanden i denne tilstand
- **Deadlocks kan altså kun opstå når:**
 - Der involveres låse/synkroniserings mekanismer, som sikre gensidig udelukkelse
 - At der indgår mere end 1 tråd



Livelocks

- **Ligner en deadlock, bortset fra at trådenes tilstand konstant ændrer sig i forhold til hinanden**
 - Dog uden at der sker noget fremskridt
 - Dette gør det livelocks svære at identificere fordi trådene faktisk udføre nogle instruktioner
- **Det svarer til, når to personer forsøger at passere hinanden i en smal gang, men ender med at blokere hinanden, fordi de hele tiden skifter side samtidig**
 - Derved forbliver de fastlåste i forhold til fremskridt, til trods for at de kontinuerligt ændre deres position mellem højre side og venstre side
- **Det vil også være tilfældet hvis alle filosoffer, som løsning til deadlock situationen, slap deres gafler på samme tid**

kan løses ved at bruge Random tid i Sleep fx `Sleep(random.Next(min, max))`

Starvation

- Betyder at en tråd ikke får adgang til en ressource
 - Kan fx forekomme ved både dead- og livelocks
- De to locks forhindrer nemlig tråden i at blive færdig, man kalder dette tilfælde for starvation, idet tråden ikke får den ressource den venter på og dermed "sulter"
- Kan også forekomme hvis andre tråde altid kommer foran i køen selvom der ikke er locks
 - F.eks som følge af forkert prioritering eller tråde der æder så mange CPU resurser at der ikke er nok til at lave et kontekst skift mellem tråde
 - Kan med særligt ske ved busy waiting (der ikke indeholder sleep)
 - F.eks som i Peterson's algoritme

Hold tungen lige i munden!

- Som det fremgår kan tråde lave handlinger, der ikke er atomare
 - Når dette sker på delte ressourcer kan det føre til at data ender i en inkonsistent state/tilstand pga. race conditions
- Vi kan forhindre at flere har adgang til ressourcerne samtidigt vha. synkroniseringsmekanismer
 - Synkroniseringsmekanismerne kan til gengæld medføre dead og live locks og ende med starvation
- Som programmører skal vi derfor kende til disse risici og holde tungen lige i munden, når vi programmere noget der anvender tråde!

Øvelser

- Gå sammen to og to og undersøg samt diskuter forskellene på de forskellige synkroniseringsmekanismer
 - Følgende er spørgsmål I forventes selv at undersøge :
 - Hvad betyder begrebet "atomar" i forhold til trådoperationer?
 - Forklar hinanden hvordan man skaber en racecondition
 - Hvornår bruger man keywordet "volatile" i C#?
 - Find eksempler på threadpool og diskuter disse
- Prøv at fremprovokere en deadlock situation ved brug af 2 mutex